Module 06 — Logging

Phase: Advanced Python
Previous: File Handling
Next: Git and GitHub

Module Outcome

Replace `print` with `logging` so a real app records **when**, **what**, and **how serious** — including exception tracebacks — in a file you can open after the crash.

By the end of this module you will be able to:

- Explain why logging beats `print` once nobody is watching the terminal
- Choose a level (DEBUG → CRITICAL) and filter with `basicConfig`
- Set `filename`, `format`, `datefmt`, and `filemode`
- Record failures with `logging.exception` inside `except`
- (Advanced) Attach a named logger with a handler and formatter
- (Advanced) Rotate logs by size (`RotatingFileHandler`) or by time (`TimedRotatingFileHandler`)

1. Why This Concept Matters

A clerk says: **"My expense wasn't saved."** `print` is gone when the window closes. You need answers:

```
When did it happen?
What operation was running?
Was it a warning, or did something fail?
```

```
print()     →  good for class / quick debug
logging     →  good for real applications
```

The built-in `logging` module writes **application flow** and **exception information** to a file (and optionally the console). Support can replay what happened. Never log passwords, OTPs, or card numbers — log that an auth attempt failed, not the secret.

2. Logging levels

Understand

Messages have a **severity**. A logger set to INFO **hides** DEBUG. The default threshold is **WARNING** if you do not configure a lower level.

Level	Value	Think of it as	Expense tracker example
CRITICAL	50	Needs high attention	Application cannot start
ERROR	40	Serious failure	CSV could not be read

Level	Value	Think of it as	Expense tracker example
WARNING	30	Caution / unusual	Daily limit almost reached
INFO	20	Important normal event	Expense added
DEBUG	10	Developer detail	Reading row 25

Remember the **meaning**, not only the numbers. NOTSET (0) means “level not set” — you will not use it in this module.

How It Works

```
import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

logging.debug("Reading row 25 — this will NOT appear at INFO level")
logging.info("Expense added: ₹120 Tea")
logging.warning("Daily limit almost reached")
logging.error("Failed to parse amount: 'abc'")
```

Expected Output:

```
INFO Expense added: ₹120 Tea
WARNING Daily limit almost reached
ERROR Failed to parse amount: 'abc'
```

DEBUG is filtered. Raise the threshold to `WARNING` and INFO disappears too.

⚠️ WATCH OUT

If you do not pass `level=`, only **WARNING and higher** appear. Students often think “I called `logging.info` so it must show” — it will not, until you set `level=logging.INFO` (or DEBUG).

Try It

Set `basicConfig` to `DEBUG` and log one line at DEBUG, INFO, WARNING, ERROR, and CRITICAL. How many lines appear? Then set the level to `WARNING` and run again. Which two disappear?

---

3. `basicConfig` — file, level, format

Understand

`logging.basicConfig(...)` configures the **root** logger once per process:

- `filename` — log file (omit it → console)
- `level` — lowest level to keep
- `format` — how each line looks
- `datefmt` — how `%(asctime)s` is rendered
- `filemode` — `'a'` append (default) or `'w'` overwrite

After configuration, write with `logging.debug` / `info` / `warning` / `error` / `critical`.

How It Works

```
import logging
import os

os.makedirs("logs", exist_ok=True)

logging.basicConfig(
    filename="logs/demo.log",
    level=logging.WARNING,
    format="%(levelname)s:%(message)s",
    filemode="w",
)

logging.debug("Debug Information")
logging.info("info Information")
logging.warning("warning Information")
logging.error("error Information")
logging.critical("critical Information")

with open("logs/demo.log", encoding="utf-8") as f:
    print(f.read(), end="")
```

Expected Output:

```
WARNING:warning Information
ERROR:error Information
CRITICAL:critical Information
```

Useful format pieces:

```
%(levelname)s   →  INFO, WARNING, ...
%(message)s     →  the text you passed
%(asctime)s     →  timestamp (shape from datefmt)
%(name)s        →  logger name (root unless you use getLogger)
```

Timestamp style (output times will differ on your machine):

```
logging.basicConfig(
    format="%(asctime)s:%(levelname)s:%(message)s",
    datefmt="%d/%m/%Y %I:%M:%S %p",
)
```

```
14/09/2026 06:30:08 PM:WARNING:Daily limit almost reached
```

`%I` is 12-hour clock; `%H` is 24-hour.

⚠️ WATCH OUT

```
# Wrong — second basicConfig in the same process usually does nothing
logging.basicConfig(level=logging.DEBUG)
logging.basicConfig(filename="other.log")   # ignored
```

```
# Prefer — configure once in main, then only log
# Restart the interpreter if a second call seems to do nothing
```

Configure logging in `main`, not at import time inside a library.

Try It

Write WARNING and higher to `logs/try.log` with `filemode="w"` and format `%(levelname)s %(message)s`. Log DEBUG and ERROR. Open the file — only ERROR (and above) should be there. Why is DEBUG missing?

---

4. Append vs overwrite — `filemode`

Understand

By default, data is **appended** to the log file. Yesterday's trail stays. Use `filemode='w'` when you want a **fresh** file for a demo or a test run.

How It Works

```
import logging
import os

os.makedirs("logs", exist_ok=True)

# This snippet assumes a fresh interpreter (basicConfig once).
logging.basicConfig(
    filename="logs/session.log",
    level=logging.INFO,
    format="%(levelname)s %(message)s",
    filemode="w",
)
logging.info("session started")

with open("logs/session.log", encoding="utf-8") as f:
    print(f.read(), end="")
```

Expected Output:

```
INFO session started
```

Change `filemode` to `"a"` (or omit it) and run the same script twice: you get two `session started` lines.

⚠ WATCH OUT

`filemode='w'` on a shared `app.log` wipes production history. Everyday apps **append**.

Try It

Run a tiny script that logs `INFO hello` with `filemode="a"` twice (two interpreter starts). How many lines in the file?

---

5. Writing exceptions — `logging.exception`

Understand

`logging.exception(msg)` belongs **inside** an `except` block. It logs at **ERROR** and includes the **traceback**. The user can still see a short print; support gets the full stack in the file.

Call it only where an exception is active (so a traceback exists).

How It Works

```
import logging
import os

os.makedirs("logs", exist_ok=True)

logging.basicConfig(
    filename="logs/mylog.txt",
    level=logging.INFO,
    format="%(levelname)s:%(message)s",
    filemode="w",
)

pairs = [("10", "2"), ("10", "0"), ("a", "2")]
for x_raw, y_raw in pairs:
    logging.info("A new request came")
    try:
        x = int(x_raw)
        y = int(y_raw)
        print("The Result:", x / y)
    except ZeroDivisionError as msg:
        print("cannot divide with zero")
        logging.exception(msg)
    except ValueError as msg:
        print("Please provide int values only")
        logging.exception(msg)
    logging.info("Request processing completed")
```

Expected Output (console):

```
The Result: 5.0
cannot divide with zero
Please provide int values only
```

`logs/mylog.txt` keeps INFO lines plus ERROR and a traceback for each failure. Open the file after you run — you should see `ZeroDivisionError` and `ValueError` stacks.

⚠ WATCH OUT

```
# Wrong — no active exception; traceback is missing or misleading
logging.exception("something went wrong")

# Prefer inside except
except ValueError:
    logging.exception("bad input")
```

`logging.error("...")` is ERROR **without** a traceback. Use `exception` when you are in `except`.

Try It

Wrap `int("abc")` in `try` / `except ValueError`. Print a short message for the user. Call `logging.exception("bad input")`. Confirm the traceback is in the log file.

---

6. Why `basicConfig` "fails" the second time

Understand

If you do not create your own logger, the **root** logger is used. The first `basicConfig` that actually installs a handler **wins**. A later call is ignored.

If module `student` configures root at import, then `test.py` calls `basicConfig` again, only the first file gets lines. That is why tiny scripts are fine with `basicConfig`, and larger apps move to a **named** logger (next section).

How It Works

```
student.py calls basicConfig → student.log     (root now has a handler)
test.py    calls basicConfig → ignored
test.py    logging.debug(...)  → still follows student.py's config
```

Configure once, in `main`, before other modules log.

⚠ WATCH OUT

Do not call `basicConfig` at import time in a helper module. The import order will freeze the wrong file or level.

Try It

In one interpreter session, call `basicConfig(level=logging.ERROR)` then `basicConfig(level=logging.DEBUG)` then `logging.info("hi")`. Does `hi` appear? Restart and put DEBUG first.

---

7. Advanced — named loggers (after `basicConfig` is solid)

`logging.info(...)` is enough for a tiny script. A larger app has parts (user, payment, files). You may want a **name** on each line, a **file and the console**, or different levels per destination.

Understand

```
Logger  →  creates the record
Handler →  decides where it goes (console, file)
Formatter →  decides how the line looks
```

`logging.getLogger("expense")` creates or **retrieves** a logger with that name. The same name twice → the **same** object (modules can share config).

Logger level is the first gate. Set the logger to the **lowest** level you care about; each handler can filter further.

Logger	Handler	Result
DEBUG	INFO	INFO and higher reach that handler
INFO	DEBUG	DEBUG never reaches the handler

Child names (`app.payment`) form a hierarchy. Children **propagate** to parents (and root) unless `propagate = False`. Duplicate lines usually mean handlers added twice, or propagate still `True` while a parent also has handlers.

How It Works

One named logger, file handler, formatter:

```
import logging
import os

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("app")
logger.setLevel(logging.DEBUG)
logger.handlers.clear()
logger.propagate = False

file_handler = logging.FileHandler("logs/app.log", mode="w", encoding="utf-8")
file_handler.setFormatter(
    logging.Formatter("%(name)s - %(levelname)s - %(message)s")
)
logger.addHandler(file_handler)

logger.info("Application started")
logger.error("Something went wrong")

with open("logs/app.log", encoding="utf-8") as f:
    print(f.read(), end="")
```

Expected Output:

```
app - INFO - Application started
app - ERROR - Something went wrong
```

Console WARNING+ and file DEBUG+ (same logger, two handlers):

```
import logging
import os

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("app")
logger.setLevel(logging.DEBUG)
logger.handlers.clear()
logger.propagate = False

fmt = logging.Formatter("%(levelname)s - %(message)s")

console_handler = logging.StreamHandler()
console_handler.setLevel(logging.WARNING)
console_handler.setFormatter(fmt)

file_handler = logging.FileHandler("logs/app.log", mode="w", encoding="utf-8")
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(fmt)

logger.addHandler(console_handler)
```

```
logger.addHandler(file_handler)

logger.debug("Debug information")
logger.info("Application started")
logger.warning("Low disk space")
logger.error("Database error")
```

Expected Output (console):

```
WARNING — Low disk space
ERROR — Database error
```

`logs/app.log` also has the DEBUG and INFO lines.

Reusable setup — attach handlers **once**:

```
import logging
import os


def get_logger(name):
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)
    if not logger.handlers:
        os.makedirs("logs", exist_ok=True)
        handler = logging.FileHandler("logs/app.log", encoding="utf-8")
        handler.setFormatter(
            logging.Formatter("%(asctime)s — %(name)s — %(levelname)s — %(message)s")
        )
        logger.addHandler(handler)
        logger.propagate = False
    return logger
```

Call `get_logger("user_service")` and `get_logger("payment_service")`. `%(name)s` shows which part wrote the line.

Privacy. Never log passwords, OTPs, full card numbers, Aadhaar, or session tokens.

⚠ WATCH OUT

```
# Wrong — addHandler on every call duplicates every line
logger.addHandler(file_handler)
logger.addHandler(file_handler)

# Prefer
if not logger.handlers:
    logger.addHandler(file_handler)
logger.propagate = False
```

Try It

Build logger `atm`: INFO+ → `atm.log`, WARNING+ → console. Log one INFO and one WARNING. Confirm INFO is in the file only.

---

8. Rotating files — size vs midnight

Understand

The expense tracker writes a line on every add. Staff run it all week. A plain `FileHandler` (or `basicConfig(filename=...)`) **never starts a new file**. `logs/app.log` grows until the disk fills — and when someone asks “what happened yesterday?”, you have one giant file.

Same Logger → Handler → Formatter chain. Swap the handler:

Need	Handler
Busy CLI / API — cap **size**	`RotatingFileHandler`
Daily audit — one file per **day**	`TimedRotatingFileHandler`

Scenario (expense desk): Keep the last few chunks of `expense.log` so a busy day cannot fill the disk. Also keep **seven calendar days** of `daily.log` so support can open yesterday’s file without searching a month of mixed lines.

How It Works

Size — when `expense.log` would pass `maxBytes`, Python renames it to `expense.log.1` (old `.1` becomes `.2`, …) and starts a fresh `expense.log`. `backupCount=3` keeps three backups; older files are deleted.

```
import logging
import os
from logging.handlers import RotatingFileHandler

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("expense_size")
logger.setLevel(logging.INFO)
logger.handlers.clear()
logger.propagate = False

handler = RotatingFileHandler(
    "logs/expense.log",
    maxBytes=200,
    backupCount=3,
    encoding="utf-8",
)
handler.setFormatter(logging.Formatter("%(levelname)s %(message)s"))
logger.addHandler(handler)

for n in range(1, 21):
    logger.info("Expense added: line %s", n)

print(sorted(name for name in os.listdir("logs") if name.startswith("expense.log")))
```

Expected Output:

```
['expense.log', 'expense.log.1', 'expense.log.2']
```

(Class `maxBytes=200` is tiny so rotation happens in one run. Production uses something like `maxBytes=5_000_000` — about 5 MB.)

Time — at midnight, today's `daily.log` is renamed with a date suffix (`daily.log.2026-09-14`). New lines go to a fresh `daily.log`. `backupCount=7` keeps a week.

```
import logging
import os
from logging.handlers import TimedRotatingFileHandler

os.makedirs("logs", exist_ok=True)

logger = logging.getLogger("expense_daily")
logger.setLevel(logging.INFO)
logger.handlers.clear()
logger.propagate = False

handler = TimedRotatingFileHandler(
    "logs/daily.log",
    when="midnight",
    interval=1,
    backupCount=7,
    encoding="utf-8",
)
handler.setFormatter(logging.Formatter("%(levelname)s %(message)s"))
logger.addHandler(handler)

logger.info("Expense tracker started")

with open("logs/daily.log", encoding="utf-8") as f:
    print(f.read(), end="")
```

Expected Output (this run — rotation has not happened yet):

```
INFO Expense tracker started
```

After several midnights the folder looks like:

```
daily.log                ← today
daily.log.2026-09-13     ← yesterday
daily.log.2026-09-12
...
```

(`when="midnight"` does not wait in class. The first snippet already proves rotation; this one is the audit-file pattern.)

⚠️ WATCH OUT

Do not attach a `FileHandler` and a rotating handler to the **same path**. They will fight over `expense.log`.

`backupCount` is a **limit**, not a guarantee of forever. Size rotator: last N chunks. Timed rotator: last N intervals. Older files are deleted.

Try It

Run the size example. Print `os.listdir("logs")`. Open `expense.log` and `expense.log.1` — newest lines are in `expense.log`.

Step-by-Step: Build It

You already have CSV + JSON from File Handling. Add a trail with `basicConfig`. (Named logger is stretch — section 7.)

```
import csv
import json
import logging
import os

os.makedirs("data", exist_ok=True)
os.makedirs("logs", exist_ok=True)

logging.basicConfig(
    filename="logs/app.log",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    datefmt="%d/%m/%Y %H:%M:%S",
    filemode="w",
)

config = {
    "app_name": "ExpenseTracker",
    "currency": "INR",
    "max_daily_limit": 5000,
    "categories": ["food", "travel", "rent"],
}
with open("data/config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

rows = [
    {"date": "2026-09-04", "category": "food", "amount": "120", "note": "Tea"},
    {"date": "2026-09-04", "category": "travel", "amount": "50", "note": "Metro"},
]
with open("data/expenses.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["date", "category", "amount", "note"])
    writer.writeheader()
    writer.writerows(rows)
logging.info("Wrote data/expenses.csv")

total = 0
with open("data/expenses.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(f"{row['note']}: ₹{row['amount']}")
        total += int(row["amount"])

with open("data/config.json", "r", encoding="utf-8") as f:
    limit = json.load(f)["max_daily_limit"]

logging.info("Day total ₹%s (limit ₹%s)", total, limit)
print("Day total ₹%s (limit ₹%s)" % (total, limit))
print("Check logs/app.log")
```

Expected Output (console):

```
Tea: ₹120
Metro: ₹50
Day total ₹170 (limit ₹5000)
Check logs/app.log
```

Open `logs/app.log` — you should see two INFO lines (write + day total). Then open the kit: 01_python_expense_tracker — add logging to the expense CLI. Do not copy `solution/`.

Practice Exercises

Attempt each problem before reading the model answer.

1. Configure `basicConfig` at DEBUG to `demo.log` with `filemode="w"` and format `%(levelname)s %(message)s`. Log one message at each of DEBUG, INFO, WARNING, ERROR, CRITICAL. How many lines?

Hint: DEBUG as the threshold keeps every standard level.

Model answer:

```
import logging

logging.basicConfig(
    filename="demo.log",
    level=logging.DEBUG,
    format="%(levelname)s %(message)s",
    filemode="w",
)
logging.debug("d")
logging.info("i")
logging.warning("w")
logging.error("e")
logging.critical("c")
```

Expected Output (`demo.log`): five lines (DEBUG through CRITICAL).

2. Repeat with `level=logging.WARNING`. Which messages disappear?

Model answer: DEBUG and INFO are filtered out. Three lines remain (WARNING, ERROR, CRITICAL).

3. Format messages as `%(asctime)s | %(levelname)s | %(message)s` with `datefmt="%Y-%m-%d %H:%M:%S"`. Log one WARNING. Confirm the line has a date, a level, and the message.

Model answer: Pass `format=` and `datefmt=` to `basicConfig`. The clock values will match **your** run time; the **shape** is `YYYY-MM-DD HH:MM:SS | WARNING | ...`.

4. Wrap `int("xyz")` in `try` / `except ValueError` and call `logging.exception` on failure. Confirm the traceback is in the log file.

Model answer:

```
import logging

logging.basicConfig(
    filename="demo.log",
    level=logging.INFO,
    format="%(levelname)s:%(message)s",
    filemode="w",
)
try:
    int("xyz")
except ValueError:
    logging.exception("bad input")
```

`demo.log` starts with `ERROR:bad input` and then a `ValueError` traceback.

5. Build logger `atm`: INFO+ → `atm.log`, WARNING+ → console. Log one INFO and one WARNING.

Model answer:

```
import logging

logger = logging.getLogger("atm")
logger.setLevel(logging.INFO)
logger.handlers.clear()
logger.propagate = False
fmt = logging.Formatter("%(levelname)s %(message)s")
fh = logging.FileHandler("atm.log", mode="w", encoding="utf-8")
fh.setLevel(logging.INFO)
ch = logging.StreamHandler()
ch.setLevel(logging.WARNING)
fh.setFormatter(fmt)
ch.setFormatter(fmt)
logger.addHandler(fh)
logger.addHandler(ch)
logger.info("ATM session started")
logger.warning("Cash drawer low")
```

Expected Output (console):

```
WARNING Cash drawer low
```

`atm.log` has both INFO and WARNING.

6. Write `get_logger(name)` that attaches one file handler (once). From two names, log as `user_service` and `payment_service` into the same file. Confirm `%(name)s` differs on each line.

Hint: `if not logger.handlers:` before `addHandler`. Set `propagate = False`.

Model answer: Shared `get_logger` attaches one file handler per name (once). Callers use `get_logger("user_service")` and `get_logger("payment_service")` so `%(name)s` shows which module wrote the line. Both can point at `logs/app.log`.

Lab

Combine `basicConfig`, INFO for success, and `logging.exception` for a bad amount. This is not the full expense-tracker ticket.

Problem

A one-function adder must leave a trail: success in the log, and a traceback when the amount is not an int.

What to Build

A short script that tries two amounts (`"40"` and `"abc"`), prints a clear message for the user, and writes `logs/lab.log`.

Requirements

- `os.makedirs("logs", exist_ok=True)`
- `basicConfig` to `logs/lab.log`, level INFO, format `%(levelname)s %(message)s`, `filemode="w"`

- For each raw amount: `int(...)` in `try`; on success `logging.info` and print `Added: <n>`
- On `ValueError`: print `Bad amount` and `logging.exception("parse failed")`

Expected Behavior

Console shows one added line and one `Bad amount`. The log file has INFO plus an ERROR traceback. No secrets in the log.

Sample Run

```
Added: 40
Bad amount
```

Constraints

Root `logging.info` / `logging.exception` is enough. Named loggers are optional bonus.

Bonus

Switch to `getLogger("lab")` + `FileHandler` with the same format.

Mini-Project (Assignment): DailyExpenseLogger

Why this matters: The File Handling tracker already saves CSV/JSON. Support still cannot answer "when did the add fail?" until every operation leaves a line in `logs/app.log`.

Lab kit: 15_Lab_Assets/01_python_expense_tracker/

This is **not** the in-class assemble. Same product; now every path logs.

Objective

Keep the expense CLI. Successful adds are INFO. Bad / missing files are ERROR (use `logging.exception` at least once). Logs go to a file, not only `print`.

Story / Scenario

Employee Management Portal finance — same CSV/JSON tracker as File Handling; every operation leaves a trail so a clerk's "it didn't save" can be replayed.

Requirements

Piece	Detail
`logs/app.log`	Prefer a **named** custom logger once `basicConfig` works
Levels	INFO for success; ERROR for failures
Resilience	Bad / missing files → log + clear message
Secrets	No passwords / OTPs / card numbers in logs
`solution/`	Do **not** copy this

Expected Behavior

- Adding a valid expense writes INFO

- Invalid amount does not crash; ERROR (or `exception`) is in the log
- You can open `logs/app.log` after the run

Sample Run

Use the kit's CLI / `main` after you add logging. Check the log file, not only the console.

Bonus Challenges

Advanced folder:

```
student_app/
├── main.py
├── student_service.py
├── logging_config.py
└── logs/student_app.log
```

- `logging_config.py` — reusable `get_logger` (console WARNING+, file DEBUG+, optional rotating handler)
- `student_service.py` — `add_student`, `find_student`, `delete_student` with INFO / WARNING / ERROR / DEBUG
- `main.py` — call the three operations so you can compare console vs file

Done when

- [ ] Logging writes to a log file (not only `print`)
- [ ] At least one `logging.exception` (or ERROR) path for bad input / missing file
- [ ] No secrets in logs
- [ ] Named logger used if you took the stretch

Interview Q&A

Q1. Logging vs `print`?
Logging gives levels, timestamps, handlers (file/console), and filtering. `print` is ad-hoc. On a server, nobody is watching the terminal.

Q2. Standard levels (high → low)?
CRITICAL (50), ERROR (40), WARNING (30), INFO (20), DEBUG (10). Default threshold is WARNING if unset.

Q3. What does `basicConfig` do? Why does a second call often fail?
It configures the root logger (file, level, format, …). Once the root logger has handlers, later `basicConfig` calls are ignored.

Q4. Why prefer `getLogger` over only root?
You can attach multiple handlers, set per-module names, and avoid the "config is frozen after first import" trap of root `basicConfig`.

Q5. Logger vs Handler vs Formatter?
Logger creates the record; Handler chooses destination; Formatter shapes the text.

Q6. `logging.error` vs `logging.exception`?
Both are ERROR. `exception` also records the current traceback — use it inside `except`.

Q7. What is `propagate`? When set it to `False`?
Child loggers forward records to parents by default. Set `propagate = False` when the child has its own handlers so lines are not duplicated.

Q8. File handler DEBUG, console handler WARNING — what appears where?
DEBUG and INFO go to the file only (if the logger allows them). WARNING and above go to both.

Q9. What must never go in log files?
Passwords, OTPs, full card numbers, Aadhaar, session tokens. Log that an auth attempt failed, not the secret.

Q10. How do you rotate logs that grow too large?
`RotatingFileHandler` when you cap **size** (`maxBytes` + `backupCount`). `TimedRotatingFileHandler` when you want one file per **interval** (`when="midnight"`, keep 7 days). Same logger chain — only the handler changes.

Cheat Sheet

```
logging.basicConfig(...)     → configure root once (file, level, format, filemode)
logging.info / warning / ... → write at that level
logging.exception(msg)       → ERROR + traceback (inside except only)
filemode "a" / "w"           → append (default) / overwrite
getLogger("app")             → named logger (same name → same object)
FileHandler / StreamHandler  → file / console
RotatingFileHandler          → new file when size hits maxBytes
TimedRotatingFileHandler     → new file at midnight (or other when=)
Formatter("...")             → line shape
propagate = False            → do not duplicate up to parent/root
```

Default threshold is WARNING. Never log secrets.

Completion Checklist

```
[ ] I can explain why logging beats print
[ ] I can set levels and see DEBUG filtered at INFO
[ ] I can use basicConfig with filename, format, filemode
[ ] I can call logging.exception inside except
[ ] I know a second basicConfig is usually ignored
[ ] I can attach a named logger + handler + formatter (Advanced)
[ ] I can choose RotatingFileHandler (size) vs TimedRotatingFileHandler (midnight)
[ ] I completed the lab
[ ] I attempted DailyExpenseLogger
[ ] I can explain Logger vs Handler vs Formatter
```

What's Next?

You can now keep a timestamped trail of success and failure — including stack traces — without relying on the terminal.

Next on the calendar is Git and GitHub so this code and its `logs/` / `data/` layout can be versioned. Database work (Python talking to MySQL) is a later bridge after the MySQL phase.

Optional, not this week: Professional Python Deep Dive.